Foundations of Distributed Systems
Exercise 7 Solutions
Differential Privacy

Eshan , Gopal, Harshit, Anna Diack

December 2, 2025

Instructions

This document contains my written answers for all tasks of Exercise 7. The Python code is provided in separate files as required. All explanations are written in my own words.

1 Task 1: Laplace Mechanism and Counting Query

Code Summary

To answer the first task, I wrote a small program that applies the Laplace mechanism to a simple counting query ("How many people in the dataset are older than 29?"). The idea is that we first compute the real value and then add controlled random noise to it so that the final result protects each individual's privacy.

The script starts with a function called `laplace_mech`. This function takes three arguments: the true value of the query, the sensitivity of the query, and the privacy budget, ε. For a counting query, the sensitivity is always 1 because adding or removing a single person can only change the count by one. The function then computes the scale parameter of the Laplace distribution by dividing the sensitivity by ε. Based on this, it draws one noise value from a Laplace distribution centered at zero and adds this noise to the true value. The idea is that the noise is large enough to mask the contribution of any one person but still small enough that the overall statistic remains useful.

The actual differential privacy-protected query is performed by the second function, `dp_count_over_29`. It starts by calculating the true count using the column "Age" from the data set. After that, it calls the Laplace mechanism with sensitivity = 1 and $\varepsilon = \log(2)$, as required in the task. The function then returns the noisy result.

At the bottom of the script, in the `__main__` section, the dataset `adult_with_pii.csv` is loaded. The code then runs the DP counting function and prints the final noisy value. Every time you run the program, the result will look slightly different because of the added randomness, which is exactly what differential privacy relies on.

Overall, the code is fairly short, but it demonstrates the essential steps of applying the Laplace mechanism: compute the true answer, pick the correct sensitivity, add Laplace noise, and then return the perturbed value.

Explanation

A counting query changes by at most one when a single individual enters or leaves the dataset, so its global sensitivity is 1. I compute the true count of rows with `Age > 29` and then add Laplace noise with scale $\frac{1}{\varepsilon}$. Since the exercise specifies $\varepsilon = \ln 2$, the mechanism achieves the expected privacy guarantee.

2 Task 2: Differentially Private Contingency Tables

Code Summary

For this task, we had to build a differentially private contingency table for any two columns of a dataset. Then, we had to apply the method to the adult data for the attributes *Relationship* and *Race*.

The idea behind the code is simple: A contingency table is basically a grid of counts. Each cell just tells us how many rows fall into a specific combination of values, like *Relationship = Husband* and *Race = White*. For a non-private table, we can simply use `pandas.crosstab`, which does exactly that.

To make the table differentially private, we keep the original structure but add Laplace noise to every single cell. Since each entry in the table is itself just a counting query, its sensitivity is 1 (changing one person can only increase or decrease a single cell count by one). That's why the same Laplace mechanism from Task 1 still applies here.

In the implementation, after computing the real table, I loop through all cells and replace each value with a noised version. The noise scale is $1/\varepsilon$, following the Laplace mechanism formula. The end result is a table with the same shape and labels, but the numbers are slightly "wiggly" due to the noise. This protects individuals while keeping the overall structure of the distribution.

For the required example, I plugged in $\varepsilon = 0.3$ and used the columns *Relationship* and *Race* from the provided Adult dataset.

(a) Parallel Composition

Parallel composition does not apply because all cells of the contingency table depend on the same underlying dataset. Even though each individual contributes to only one cell, the mechanism is not operating on disjoint subsets of the data. Therefore, the table uses sequential composition.

(b) Privacy Cost and Accuracy

The number of cells matters for accuracy, because each cell receives noise while cell counts become smaller as the table becomes more detailed. The total privacy cost depends on how we distribute ε. In this exercise, we assign the full ε to the table as specified.

3 Task 3: Most Common Occupation

Code Summary

This task is about selecting the most common occupation in a differentially private way. The idea is similar to Task 1: we start with a normal count, add Laplace noise, and finally choose the occupation with the highest noisy value.

The scoring function is straightforward. It simply counts how many times each occupation appears in the dataset. This number is our base "score": the more common the job, the higher its score.

To make the selection private, we run the Laplace mechanism on each occupation's score. Since a single person can change the count of exactly one occupation by at most 1, the sensitivity is again equal to 1. For every possible occupation, I compute its true score, add Laplace noise drawn with scale $1/\varepsilon$, and store the result. At the end, I look at all noisy scores and pick the occupation that ended up with the highest noisy value.

This method is essentially the Laplace version of the exponential mechanism: we distort each score slightly and rely on the idea that the most frequent occupation is still likely to win, while avoiding the disclosure of the exact counts. With $\varepsilon = 0.05$, the noise is relatively strong, so the output may fluctuate between runs, but the privacy guarantee is correspondingly strong.

(a) Sensitivity

The score can change by at most one when one person changes or removes their occupation, so the sensitivity is 1.

(b) Total Privacy Cost

If we add Laplace noise to $|R|$ occupations separately, the total privacy cost is $|R|\varepsilon$ by sequential composition. Even though we only publish the final chosen occupation, the noise generation already consumes the privacy budget.

4 Task 4: Differentially Private Sum of Capital Gain

Code Summary

Summation queries are a little more delicate than counting queries. This is because a single row can contribute a very large value. To make the query differentially private, we need to ensure that the influence of any one person is bounded. The usual approach is to *clip* the values before summing them. Clipping means we cap the values in a fixed range, typically something like $[0, B]$ or $[-B, B]$, depending on the data.

Once the values are clipped, the sensitivity of the sum becomes simply the clipping bound B: if one person enters or leaves the dataset, the sum can change by at most B. With that sensitivity, we can apply the Laplace mechanism, as in the earlier tasks.

In the code, I chose the clipping bound by taking the 95th percentile of the *Capital Gain* column. This is one reasonable way to pick a bound without inspecting exact maximums, which we are not allowed to "peek" at in a privacy-sensitive scenario. After clipping, I compute the true sum and add Laplace noise, scaled by B/ε. Since the mechanism is only applied once, the total privacy cost is simply ε.

(a) Clipping Parameter

I used the 95th percentile as a clipping threshold. This avoids outlier dominance while keeping values meaningful.

(b) Sensitivity

For clipped values in $[-B, B]$, the sensitivity of the sum is B because the presence or absence of one person changes the sum by at most B.

(c) Privacy Cost

Since the mechanism is used once, the total privacy loss is exactly ε.

5 Task 5: Sensitivity of the Differencing Attack

Explanation

The differencing attack isolates the age of a specific individual, because

$$q_1 - q_2 = \text{Age of the excluded person.}$$

Thus, the sensitivity equals the maximum possible age in the dataset. Without clipping this is unbounded, which is why the attack is so powerful.

Sensitivity of the differencing attack.

Consider two neighboring datasets:

- D_1: full dataset
- D_2: dataset without one specific person

The query computes:

$$f(D_1) - f(D_2) = \text{Age of the removed person}$$

Thus, the sensitivity is:

$$\Delta f = \max(\text{Age})$$

This value is unbounded unless we clip the ages.

This aligns with the lecture slides on summation sensitivity and the risks associated with differencing attacks.

DP Version

To create a DP-safe version, I clipped ages and applied the Laplace mechanism using sensitivity equal to the clipping bound.